K8S-14: Kubernetes Deployment Guide

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 14 of 14 | **Type:** LAB | **Created:** April 2026 | **Last Updated:** 10/15/2026

Overview

Step-by-step guide to deploy Dynatrace monitoring on Kubernetes with all recommended production settings. Covers operator installation, DynaKube configuration, metadata enrichment, build propagation, segments, and verification.

This is a hands-on deployment notebook. Each section includes commands to run and queries to verify. Follow the sections in order for a complete production deployment.

Table of Contents

1. Prerequisites
2. Create Access Tokens
3. Install Dynatrace Operator
4. Configure DynaKube CR (Production Recommended)
5. Label Namespaces for Monitoring
6. Configure Telemetry Enrichment Rules
7. Configure Build Propagation & Release Detection
8. Create Segments
9. Verify Deployment
10. Monitoring-the-Monitoring
11. Recommended Next Steps
12. Summary

1. Prerequisites

Requirement	Details
Kubernetes Cluster	A version supported by your Operator release (check the Operator's supported-versions table), with `kubectl` access and cluster-admin permissions
Helm	Version 3.x installed locally
Dynatrace Tenant	SaaS environment with admin access
Network	Outbound 443 to `*.live.dynatrace.com` and `*.apps.dynatrace.com`
Knowledge	Completed K8S-01 (Fundamentals) and K8S-02 (DynaKube Deployment)

Verify Your Environment

Run these commands to confirm your cluster is ready:

```
# Verify Kubernetes version (kubectl 1.28+ removed --short)
kubectl version

# Verify Helm version (must be 3.x)
helm version --short

# Verify cluster access
kubectl get nodes
```

2. Create Access Tokens

The Dynatrace Operator uses two tokens: an **Operator token** for managing Dynatrace components in the cluster and a **Data Ingest token** for sending telemetry. The Kubernetes onboarding flow in the Kubernetes app creates both for you; the current documentation describes them as **platform tokens** on a dedicated service user.

Operator token (platform token)

Scope	Purpose
`fleet-management:activegate.connection-info:read`	ActiveGate lifecycle information
`fleet-management:activegate.tokens:create`	ActiveGate authentication tokens
`fleet-management:container-images:read`	Image information for managed components
`fleet-management:oneagent.connection-info:read`	OneAgent lifecycle information
`fleet-management:oneagents:download`	OneAgent lifecycle
`settings:objects:read` / `settings:objects:write`	Kubernetes API monitoring, KSPM and log-monitoring settings

Data Ingest token (platform token)

Scope	Purpose
`openpipeline:logs:ingest`	Send container and pod logs
`openpipeline:metrics:ingest`	Send Kubernetes and Prometheus metrics
`openpipeline:traces:ingest`	Send distributed traces
`storage:metrics:write`	Write metrics

Classic access tokens remain accepted (*"existing access tokens continue to be accepted"*), and Operators before 1.10.0 accept only them. If a scope is missing, the DynaKube status **Tokens** condition names it (`kubectl -n dynatrace describe dynakube`).

> **Sources:** Tokens and permissions (DT docs) — *"Operator token —assigned the Kubernetes Operator policy. Manages the lifecycle of all Dynatrace components in the cluster."*, Operator 1.10.0 release notes (DT docs) — *"existing access tokens continue to be accepted"*.

Save Your Tokens

Copy both tokens immediately after generation. You cannot retrieve them later.

```
# Store tokens as environment variables for the next steps
export OPERATOR_TOKEN="&lt;operator-token&gt;"
export DATA_INGEST_TOKEN="&lt;data-ingest-token&gt;"
```

3. Install Dynatrace Operator

Step 1: Install the Operator via Helm

```
helm install dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.2 \
  --create-namespace \
  --namespace dynatrace \
  --atomic
```

The `--atomic` flag ensures the installation is rolled back automatically if any component fails to start.

> **Pin `--version` so this lab is reproducible.** Without it Helm takes whatever is newest at run time, so two people running this lab a month apart get different operators. **1.10.2** (released July 30, 2026) is the recommended pin: it fixes a workload/namespace **tagging-precedence regression**, a `dynatrace-webhook` `CrashLoopBackOff` under the gVisor runtime class, and injected pods hanging on the OneAgent-binary download, and it starts logging metadata-enrichment rules it cannot apply. The tagging fix is a **behaviour change** if you have several rules for one key — see K8S-10 before upgrading a cluster that relies on the old ordering. **Skip 1.10.0** (auto-update defect; flagged `prerelease: true` on the releases page). Estates adopt operator releases on their own schedule — **1.10.1 remains a working pin** until you upgrade, with the OpenShift-manifest caveat in K8S-09 §2. Never pin below 1.4.1 (CSI liveness-probe crash-loop window — K8S-09 §2). **Operator 1.11.0** (released 10/01/2026) is the newest release: it removes the `v1beta4` DynaKube API (this lab already uses `v1beta6`) and adds image-volume code-module injection

> (K8S-12 § 2). Move the pin to 1.11.0 once you have validated it on a non-production cluster.

Step 2: Create the Token Secret

```
kubectl -n dynatrace create secret generic dynakube \
  --from-literal=apiToken=$OPERATOR_TOKEN \
  --from-literal=dataIngestToken=$DATA_INGEST_TOKEN
```

Step 3: Verify Operator Pods

```
kubectl get pods -n dynatrace
```

Expected output:

```
NAME                                  READY   STATUS    RESTARTS   AGE
dynatrace-operator-xxxxxxxxxx-xxxxx   1/1     Running   0          30s
dynatrace-webhook-xxxxxxxxxx-xxxxx    1/1     Running   0          30s
```

> **Note:** The operator and webhook pods must both be `Running` before applying the DynaKube CR. If either pod is in `CrashLoopBackOff`, check the logs with `kubectl logs -n dynatrace <pod-name>`.

4. Configure DynaKube CR (Production Recommended)

The DynaKube custom resource defines how Dynatrace monitors your cluster. The configuration below represents production-recommended settings with all key features enabled.

Complete DynaKube YAML

Save this as `dynakube.yaml` and customize the placeholder values:

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
  annotations:
    # Fail pod startup if injection fails — surfaces problems immediately
    feature.dynatrace.com/injection-failure-policy: "fail"
    # Enable build label propagation for Release Inventory
    feature.dynatrace.com/label-version-detection: "true"
    # Extend CSI mount timeout for cloud providers with slow volume attachment
    feature.dynatrace.com/max-csi-mount-timeout: "15m"
spec:
  apiUrl: https://&lt;ENVIRONMENT_ID&gt;.live.dynatrace.com/api

  # Network zone — isolates traffic per cluster
  networkZone: &lt;cluster-name&gt;

  # Metadata enrichment — flows K8s labels into all telemetry
  metadataEnrichment:
    enabled: true

  oneAgent:
    # Host group — scopes dashboards and alerts per cluster
    hostGroup: &lt;cluster-name&gt;
    cloudNativeFullStack:
      # Opt-in namespace selector
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"

      # Host properties for entity-level context
      args:
        - --set-host-property=environment=production
        - --set-host-property=team=platform
        - --set-host-property=cost-center=infrastructure

      # Tolerate control plane nodes
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists

  # ActiveGate — K8s API monitoring + routing
  activeGate:
    capabilities:
      - routing
```

```
      - kubernetes-monitoring
      - metrics-ingest      # in-cluster metrics ingest endpoint (optional)
    replicas: 2
    resources:
      requests:
        cpu: 500m
        memory: 1Gi
      limits:
        cpu: 2000m
        memory: 2Gi
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
        labelSelector:
          matchLabels:
            app.kubernetes.io/component: activegate

  # Log monitoring
  logMonitoring: {}
```

Apply the DynaKube CR

```
# Replace placeholders, then apply
kubectl apply -f dynakube.yaml
```

Setting Explanations

Setting	Value	Why It Matters
`injection-failure-policy: "fail"`	Fail pod startup on injection error	Surfaces misconfiguration immediately instead of running uninstrumented
`label-version-detection: "true"`	Propagate `app.kubernetes.io/version`	Populates Release Inventory for deployment tracking

Setting	Value	Why It Matters
`max-csi-mount-timeout: "15m"`	Extended CSI driver timeout	Prevents pod failures on cloud providers with slow EBS/disk attachment
`networkZone`	Per-cluster zone	Keeps OneAgent traffic on this cluster's ActiveGates
`metadataEnrichment.enabled`	`true`	Flows K8s labels/annotations into all telemetry signals
`cloudNativeFullStack`	Full-stack mode	Full code-level visibility with automatic injection
`namespaceSelector`	`dt-monitoring: "true"`	Only labeled namespaces get OneAgent injection — this is the opt-in
`hostGroup`	Per-cluster name	Groups hosts for scoped alerting and dashboards
`args: --set-host-property`	Custom properties	Adds environment, team, cost-center to every host entity
`tolerations`	Control plane toleration	Ensures DaemonSet runs on all nodes including control plane
`activeGate.replicas: 2`	High availability	Survives single-pod failure
`topologySpreadConstraints`	Zone-aware scheduling	Distributes ActiveGate pods across availability zones

Setting	Value	Why It Matters
`capabilities: routing`	ActiveGate routing	Routes OneAgent traffic through ActiveGate
`capabilities: kubernetes-monitoring`	K8s API integration	Enables cluster-level monitoring, events, and workload data
`capabilities: metrics-ingest`	Metrics ingest endpoint	Lets in-cluster sources send metrics to the ActiveGate. Annotation-based Prometheus scraping comes from `kubernetes-monitoring` plus the *Monitor annotated Prometheus exporters* setting (K8S-13 § 3)
`logMonitoring: {}`	Log collection enabled	Collects container stdout/stderr logs

> **Do not add `feature.dynatrace.com/automatic-injection: "false"` here.** That flag switches injection to per-pod opt-in: *"Pods that should be injected have to be annotated with oneagent.dynatrace.com/inject: "true""*. With it set, labelling namespaces alone injects nothing. Earlier versions of this lab set it; K8S-11 § 2 explains both opt-in levels.

5. Label Namespaces for Monitoring

Because the DynaKube's `namespaceSelector` matches `dt-monitoring: "true"`, only namespaces with that label receive OneAgent injection.

Label Application Namespaces

```
# Label each application namespace
kubectl label namespace &lt;app-namespace-1&gt; dt-monitoring=true
kubectl label namespace &lt;app-namespace-2&gt; dt-monitoring=true

# Verify labeled namespaces
kubectl get namespaces -l dt-monitoring=true
```

Namespaces to NOT Label

In community practice, these namespaces are left unlabelled:

Namespace	Reason
`kube-system`	Core Kubernetes components — injection can cause instability
`kube-public`	Cluster info namespace — no application workloads
`kube-node-lease`	Node heartbeat namespace — no application workloads
`cert-manager`	Certificate management — injection interferes with TLS operations
`dynatrace`	Dynatrace's own namespace — self-monitoring is handled internally
`istio-system`	Service mesh control plane — injection conflicts with Envoy sidecars

> **Tip:** After labeling, trigger a rolling restart of deployments in those namespaces to activate injection: `kubectl rollout restart deployment -n <namespace>`.

6. Configure Telemetry Enrichment Rules

Telemetry enrichment flows Kubernetes labels and annotations into all telemetry signals (logs, metrics, traces, events). This enables filtering, cost allocation, and IAM scoping.

Where the rules live

The **Kubernetes telemetry enrichment** setting, schema `builtin:kubernetes.generic.metadata.enrichment`, at environment or cluster scope.

From SaaS 1.345 you can opt into central configuration instead (K8S-10 § 9).

Recommended Enrichment Rules

#	Metadata Type	Key	Mapped To	Purpose
1	Namespace label	`team`	`dt.security_context`	Enables record-level IAM — teams see only their own data
2	Namespace label	`cost-center`	`dt.cost.costcenter`	Enables FinOps cost allocation per team
3	Namespace label	`app.kubernetes.io/part-of`	`dt.cost.product`	Enables product-level cost attribution

Label Your Namespaces

The enrichment rules map **existing** Kubernetes labels to Dynatrace fields. Ensure your namespaces carry these labels:

```
# Add organizational labels to namespaces
kubectl label namespace &lt;app-namespace&gt; team=backend
kubectl label namespace &lt;app-namespace&gt; cost-center=ecommerce
kubectl label namespace &lt;app-namespace&gt; app.kubernetes.io/part-of=checkout-platform

# Verify labels
kubectl get namespace &lt;app-namespace&gt; --show-labels
```

Propagation Timing

The schema states: *"New rules may take up to 45 minutes to take effect. Pod restarts are required after the 45 mins to ensure the changes take effect."* Treat a new namespace

label the same way — restart the pods once the rule is in effect.

> **Important:** If you create enrichment rules after DynaKube is already deployed, existing pods must be restarted to pick up the enriched metadata. Run `kubectl rollout restart deployment -n <namespace>` for affected namespaces.
>
> **Sources:** Kubernetes telemetry enrichment schema (DT docs) — *"New rules may take up to 45 minutes to take effect."*
>
> **See also:** K8S-10 (Metadata Telemetry Enrichment) for a deep dive into enrichment methods, file locations, and advanced use cases.

7. Configure Build Propagation & Release Detection

Build propagation connects your Kubernetes deployment metadata to the Dynatrace Release Inventory. This gives you deployment tracking, version comparison, and release health monitoring.

What Is Already Enabled

The DynaKube in Section 4 includes:

```
annotations:
  feature.dynatrace.com/label-version-detection: "true"
```

This tells the Dynatrace Operator to read version information from standard Kubernetes labels.

Required Pod Labels

The **pod** must carry these labels — put them in the pod template. *"Only pod labels are detected, not workload (Deployment/StatefulSet) labels."*

Pod label	Maps to	Example
`app.kubernetes.io/version`	`DT_RELEASE_VERSION`	`1.4.2`, `2025.03.15-hotfix`

Pod label	Maps to	Example
`app.kubernetes.io/part-of`	`DT_RELEASE_PRODUCT`	`checkout-platform`
`dynatrace-release-stage`	`DT_RELEASE_STAGE`	`production`

Example Deployment snippet:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
  labels:
    app.kubernetes.io/name: payment-service
    app.kubernetes.io/part-of: checkout-platform
    app.kubernetes.io/version: "1.4.2"
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: payment-service
        app.kubernetes.io/part-of: checkout-platform
        app.kubernetes.io/version: "1.4.2"
```

Optional: Custom Release Mapping

If your deployment model uses non-standard labels, add namespace annotations to override the defaults:

```
apiVersion: v1
kind: Namespace
metadata:
  name: checkout
  annotations:
    mapping.release.dynatrace.com/version: "metadata.labels['app-version']"
    mapping.release.dynatrace.com/product: "metadata.labels['product-name']"
    mapping.release.dynatrace.com/stage: "metadata.namespace"
```

Verify in Dynatrace

After deploying workloads with the required labels:

1. Open the **Release Monitoring** dashboard, or run the § 9.5 query

2. Confirm your workloads appear with version information
3. Change the version in the pod template, roll out, and confirm the new version appears

> **Sources:** Version detection methods (DT docs) — *"Only pod labels are detected, not workload (Deployment/StatefulSet) labels."*

8. Create Segments

Segments filter what a view shows across Grail data. They take over the filtering job of Management Zones; access control moves to IAM policies and boundaries (MZ2POL series).

Recommended Segments

Create segments based on the enrichment rules configured in Section 6:

Segment Name	Include condition	Use Case
Per-team: `Team: Backend`	`dt.security_context` equals `backend`	Team-scoped dashboards
Per-team: `Team: Frontend`	`dt.security_context` equals `frontend`	Team-scoped dashboards
Per-cluster: `Cluster: Production`	`k8s.cluster.name` equals `prod-us-east-1`	Cluster-level views
Per-namespace: `Namespace: Checkout`	`k8s.namespace.name` equals `checkout`	Application-scoped monitoring
Per-environment: `Environment: Staging`	`k8s.cluster.name` equals `staging-us-east-1`	Environment isolation

Creating Segments as Code

Segments are not Settings objects, so the Settings API cannot create them. They have their own **Filter Segments API** at `/platform/storage/filter-segments/v1/filter-segments`, called with a platform token. Build a segment once in the UI, read it back

through that API, and version the JSON. ORGNZ-10 covers the API, the include syntax and the permissions (`storage:filter-segments:*`).

> **See also:** ORGNZ-08 and ORGNZ-10 for segment design patterns.

9. Verify Deployment

Run these DQL queries to confirm every component is working correctly.

9.1 Verify Dynatrace Components Are Running

```
// Verify Dynatrace components are running
fetch logs, from:-1h
| filter k8s.namespace.name == "dynatrace"
| summarize count = count(), by:{k8s.pod.name}
| sort count desc
| limit 20
```

Expected: You should see pods for `dynatrace-operator`, `dynatrace-webhook`, `dynakube-oneagent-*`, and `dynakube-activegate-*`.

9.2 Verify Host Monitoring

```
// Check all monitored hosts
//
// Corrected 08/12/2026: `host_group` is not a field on dt.entity.host
(FIELD_DOES_NOT_EXIST).
// The host group lives on the Smartscape HOST node as `dt.host_group.id`; on
the classic entity it
// appears only inside the `tags` array as "Host Group:&lt;name&gt;".
smartscapeNodes "HOST"
| fieldsKeep name, dt.host_group.id
| sort name asc

// Classic equivalent (no host-group column available):
//   fetch dt.entity.host, from:-7d
//   | fieldsKeep entity.name, state, tags
//   | sort entity.name asc
// `state` is a classic-only field with no Smartscape node equivalent
(Smartscape expresses
// liveness via node lifetime).
```

Expected: every node in your cluster appears, with `dt.host_group.id` matching the `<cluster-name>` value from your DynaKube.

9.3 Verify Container Metrics Are Flowing

```
// Top containers by CPU
timeseries avgCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h, by:
{k8s.container.name, k8s.namespace.name}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

Expected: Container CPU metrics from your labeled namespaces. If this returns no data, verify namespace labels and wait for data to propagate (up to 5 minutes).

9.4 Verify Metadata Enrichment

```
// Check enrichment is working — dt.security_context should be populated
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize count = count(), by:{dt.security_context}
| sort count desc
```

Expected: Logs grouped by the `team` label you assigned to namespaces. If empty, verify enrichment rules are configured (Section 6) and wait up to 45 minutes for propagation.

9.5 Verify Build Propagation (Release Inventory)

```
// Check Release Inventory population
//
// Corrected 08/12/2026: `softwareVersion` does not exist on
dt.entity.process_group_instance
// (FIELD_DOES_NOT_EXIST). The Release Inventory fields are `releasesVersion`,
`releasesProduct`
// and `releasesStage`. Time range added — dt.entity.* returns only entities
seen in the window.
fetch dt.entity.process_group_instance, from:-7d
| filter isNotNull(releasesVersion)
| fields entity.name, releasesVersion, releasesProduct, releasesStage
| limit 10

// Note: releasesVersion renders as a structured value, e.g.
//   ReleaseVersionInfo{version='1.5.2', source=AGENT_REGISTRY, timestamp=0}
```

Expected: Process groups with version information from `app.kubernetes.io/version` labels. If empty, verify your pods carry the required labels (Section 7).

9.6 Verify ActiveGate Health

```
// ActiveGate pod CPU — should show 2 pods across zones
timeseries agCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
  filter:{k8s.namespace.name == "dynatrace" and
matchesValue(k8s.container.name, "*activegate*")},
  by:{k8s.pod.name}
| fieldsAdd avgCpu = arrayAvg(agCpu)
| fields k8s.pod.name, avgCpu
```

Expected: Two ActiveGate pods with CPU metrics. If only one pod appears, check the topology spread constraints and verify your cluster has multiple availability zones.

10. Monitoring-the-Monitoring

After deployment, set up ongoing health checks for the Dynatrace components themselves.

10.1 OneAgent Memory Usage per Node

```
// OneAgent memory consumption per node
timeseries oaMem = avg(dt.kubernetes.container.memory_working_set), from:-1h,
  filter:{k8s.namespace.name == "dynatrace" and
matchesValue(k8s.container.name, "*oneagent*")},
  by:{k8s.pod.name}
| fieldsAdd avgMemMB = arrayAvg(oaMem) / 1048576
| sort avgMemMB desc
```

There is no published healthy range. Record the baseline for your nodes after the first week and alert on sustained growth above it.

10.2 Dynatrace Component Failures

```
// Kubernetes events for Dynatrace components — restarts, back-offs, mount and
scheduling failures
// One open event is re-written about every 5 min under the same event.id, so
it can repeat here.
// event.provider is the discriminator. event.kind never takes a "K8S_EVENT"
value: these
// records carry "DAVIS_EVENT", so filtering on kind returns zero rows and
looks like health.
fetch events, from:-24h
| filter event.provider == "KUBERNETES_EVENT"
| filter k8s.namespace.name == "dynatrace"
| fields timestamp, k8s.cluster.name, k8s.workload.name,
         dt.kubernetes.event.reason,
         dt.kubernetes.event.involved_object.name,
         dt.kubernetes.event.message
| sort timestamp desc
| limit 30
```

Expected: no `Failed`, `FailedMount`, `FailedScheduling` or `BackOff` reasons. If present, increase resource limits in the DynaKube CR or follow the matching row in K8S-09 §9.

OOM kills are not an event — Dynatrace emits them as a counter derived from container status, so check the metric separately:

```
timeseries oom = sum(dt.kubernetes.container.oom_kills), from:-24h,
  by:{k8s.cluster.name, k8s.workload.name}
| fieldsAdd oomTotal = arraySum(oom)
| filter oomTotal &gt; 0
| sort oomTotal desc
```

10.3 Metric Data Gaps

```
// Hosts with gaps in CPU reporting (OneAgent interruptions)
// arraySize counts empty buckets too, so count only the non-null ones.
timeseries hostCpu = avg(dt.host.cpu.usage), from:-6h, by:{dt.entity.host}
| fieldsAdd buckets = arraySize(hostCpu), reported =
arraySize(arrayRemoveNulls(hostCpu))
| fieldsAdd completeness = round(100.0 * reported / buckets, decimals: 1)
| filter completeness &lt; 95.0
| fieldsAdd hostName = entityName(dt.entity.host, type:"dt.entity.host")
| fields hostName, completeness, reported, buckets
| sort completeness asc
// A host that stopped reporting entirely drops out of timeseries altogether —
// cross-check smartscapeNodes "HOST" for hosts with no series.
```

Expected: All hosts at 100% completeness. Hosts below 95% indicate OneAgent restarts, pod evictions, or network interruptions to the Dynatrace backend.

11. Recommended Next Steps

With the base deployment complete, these additional configurations maximize the value of your Dynatrace Kubernetes monitoring:

Priority	Configuration	What It Enables	Reference
High	OpenPipeline log routing	Route logs to buckets by namespace, set retention tiers	OPLOGS series

Priority	Configuration	What It Enables	Reference
High	Custom Grail buckets	Separate retention for prod vs. non-prod, compliance data	ORGNZ-02
High	Kubernetes anomaly detection	Built-in node, workload and namespace alerts, plus custom alerts	K8S-04 § 8
Medium	Kubernetes dashboards	Cluster health overview, workload summary, namespace drill-down	DASH series
Medium	Alerting workflows	detected problem → Slack, Teams, PagerDuty, Jira	WFLOW series
Medium	Prometheus scraping	Collect custom app metrics via pod annotations	K8S-13
Low	GitOps DynaKube management	Version-control DynaKube CR with ArgoCD or Flux	K8S-03

12. Summary

Deployment Checklist

Step	Status	Notes
1. Prerequisites verified	[ ]	Kubernetes version listed on Dynatrace Operator support and known issues (DT docs), Helm 3.x, network access
2. Access tokens created	[ ]	Operator token + Data Ingest token
3. Operator installed	[ ]	Helm chart + token secret
4. DynaKube applied	[ ]	v1beta6 with all production settings
5. Namespaces labeled	[ ]	`dt-monitoring=true` on app namespaces
6. Enrichment rules configured	[ ]	team, cost-center, part-of

Step	Status	Notes
7. Build labels added	[ ]	`app.kubernetes.io/version` on pods
8. Segments created	[ ]	Per-team, per-cluster, per-namespace
9. Deployment verified	[ ]	All DQL verification queries passing
10. Monitoring-the-monitoring	[ ]	OneAgent health, AG health, data gaps

Key Takeaways

Concept	Recommendation
Injection model	Opt-in via `namespaceSelector`; leave system namespaces unlabelled
Failure policy	`fail` — surface problems immediately, do not run uninstrumented
ActiveGate	2 replicas with zone-aware topology spread
Metadata enrichment	Enable from day one — retrofitting is expensive
Build propagation	`app.kubernetes.io/version` and `part-of` on the pod template
Segments	Filtering in Gen3, built on enriched fields; access control is IAM's job
Verification	Run DQL checks after every deployment change

Additional Resources

- Set up Dynatrace on Kubernetes (DT docs)
- Quickstart for K8s setup (DT docs)
- DynaKube parameters (DT docs)
- Dynatrace Operator Helm chart (Dynatrace GitHub)

- Dynatrace Operator releases (Dynatrace GitHub)
- Kubernetes monitoring overview (DT docs)
- Metadata enrichment (DT docs)
- Operator + DynaKube troubleshooting (DT docs)
- Segments (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.